chance of testing what you actually want to test. For example, actually starting a thread can be quite a time-consuming process, so if you didn't have the threads wait for the `go` signal, then the push thread may have completed before the pop thread even started, which would completely defeat the point of the test. Using the futures in this way ensures that both threads are running and blocked on the same future. Unblocking the future then allows both threads to run. Once you're familiar with the structure, it should be relatively straightforward to create new tests in the same pattern. For tests that require more than two threads, this pattern is readily extended to additional threads.

So far, we've just been looking at the *correctness* of multithreaded code. Although this is the most important issue, it's not the only reason you test: it's also important to test the *performance* of multithreaded code, so let's look at that next.

10.2.6. Testing the performance of multithreaded code

One of the main reasons you might choose to use concurrency in an application is to make use of the increasing prevalence of multicore processors to improve the performance of your applications. It's therefore important to actually test your code to confirm that the performance does indeed improve, just as you'd do with any other attempt at optimization.

The particular issue with using concurrency for performance is the *scalability*—you want code that runs approximately 24 times faster or processes 24 times as much data on a 24-core machine than on a single-core machine, all else being equal. You don't want code that runs twice as fast on a dual-core machine but is actually slower on a 24-core machine. As you saw in section 8.4.2, if a significant section of your code runs on only one thread, this can limit the potential performance gain. It's therefore worth looking at the overall design of the code before you start testing, so you know whether you're hoping for a factor-of-24 improvement, or whether the serial portion of your code means you're limited to a maximum of a factor of 3.